COMP 201 – Spring 2026

Assignment 2: Heap Management

Assigned: 13 March 2026 Due: 1 April 2026 23:59

Lead TA: M. Burak Kizil (*mkizil19@ku.edu.tr*)

---

1 Introduction

For centuries, the **Bureau of Temporal Affairs (BTA)** has maintained absolute control over the universe, ensuring that history follows a single, unaltered path known as the **Prime Timeline**. However, anomalies are causing the Prime Timeline to fracture into alternative universes, each generating its own historical events.

In this assignment, you will build a **Timeline Simulator** to model this multiverse. You will develop a system that reads a simulation script, dynamically constructs a hierarchy of timelines, records historical events, and completely obliterates unstable branches when commanded by the Bureau.



2 System Architecture

The multiverse is modeled as a **dynamically expanding N-ary Tree**.

- **Timeline (The Node):** Each timeline in the tree represents a distinct universe. It maintains two dynamically growing lists:
  - A list of historical **Events** that occurred within it.
  - A list of **Child Timelines** (branches) that spawned from it.
- **Dynamic Expansion:** Timelines do not know in advance how many events they will experience or how many branches they will spawn. They start with an initial capacity. Once this capacity is reached, the timeline must dynamically expand its storage to accommodate new events or branches.

3 Command Specifications

Your simulator will process commands to manipulate the multiverse. You must implement the logic to handle the following operations smoothly:

- `INIT <timeline_id>`
  Logic: Initializes the Prime Timeline (the root of your tree). This will always be the first command in a simulation file.
- `ADD_EVENT <timeline_id> <event_id> "<description>" <impact_score>`
  Logic: Records a new historical event inside the specified timeline.
  Constraint: The event description is provided inside double quotes and may contain spaces. Your program must extract the exact string between the quotes and store a completely independent copy of it in memory. If the timeline's event storage is full, it must be expanded before adding the event.
- `BRANCH <parent_timeline_id> <new_timeline_id>`
  Logic: Creates a new child timeline branching off from the specified parent timeline. If the parent's branch storage is full, it must be expanded. The new branch must correctly link back to its parent.
- `PRUNE <timeline_id>`
  Logic: The Central Committee has ordered the deletion of a timeline. You must sever this timeline from its parent.
  Constraint: When removing a branch from the parent's list, you must shift the remaining branches to close the gap. Leaving empty slots in the middle of the list is not allowed. After severing the connection, the target timeline and all of its sub-branches must be completely destroyed.
- `PRINT`
  Logic: Outputs the current state of the multiverse to the terminal. It should start from the Prime Timeline and recursively print all events and sub-branches with proper indentation to show the tree depth.

4 Output Format (PRINT Command)

When the `PRINT` command is called, you must output the exact state of the tree. The ordering must follow a **Depth-First Search (DFS)** traversal. The timeline is printed first, followed by its events (in the order they were added), and then its branches (in the order they were created), recursively.

Indentation Rule: Every level of depth adds two spaces (`"  "`) of indentation.

Exact Example Output:

```
--- BTA TIMELINE REPORT ---
Timeline [0]:
  Event [101]: The Bureau is Founded (Impact: 100)
  Event [102]: First Time Machine Built (Impact: 80)
  -> Branch:
    Timeline [1]:
      Event [201]: Minor Anomaly in London (Impact: 30)
      -> Branch:
        Timeline [2]:
          Event [301]: Cyberpunk Renaissance (Impact: 90)
  -> Branch:
    Timeline [3]:
      Event [401]: Ice Age Returns (Impact: -50)
---------------------------
```

5 File I/O & Execution

Your program will not be tested via an interactive shell. Instead, it must read commands sequentially from a text file provided via command-line arguments.

Execution Format:

```
./bta_simulator <simulation_file.txt>
```

Parsing Rules & Limits:

- **Maximum Line Length:** 1024 characters.
- **Maximum Description Length:** 512 characters.
- **Integer Ranges:** IDs (`timeline_id`, `event_id`) are non-negative 32-bit integers. Impact scores are 32-bit signed integers (can be negative).
- Your program must read the file line by line. Lines may end with varying newline characters (e.g., `\n` or `\r\n`). You are responsible for sanitizing the input.
- Empty lines, or lines starting with a `#` (comments), must be safely ignored.
- If a command references a timeline ID that does not exist, your program should print an error message and continue to the next line without crashing.

6 Memory Management Rules

The BTA mainframe has strictly **zero tolerance for memory leaks**.

- **Allocation:** Every new timeline, event, and event description string must be dynamically allocated.
- **Total Annihilation:** When a `PRUNE` command is executed, or when the simulation ends, you must perform a recursive cleanup. Every single dynamically allocated byte—including description strings, event structures, branch arrays, and timeline nodes—must be successfully released back to the system.
- **Evaluation:** Your submissions will be thoroughly tested using memory profiling tools (such as Valgrind). Memory leaks, dangling pointers, or invalid memory accesses will result in a heavy penalty.

7 Memory Management Rules

The BTA mainframe has strictly **zero tolerance for memory leaks**.

- **Allocation:** Every new timeline, event, and event description string must be dynamically allocated.
- **Total Annihilation:** When a `PRUNE` command is executed, or when the simulation ends, you must perform a recursive cleanup. Every single dynamically allocated byte—including description strings, event structures, branch arrays, and timeline nodes—must be successfully released back to the system.
- **Evaluation:** Your submissions will be thoroughly tested using memory profiling tools (such as Valgrind). Memory leaks, dangling pointers, or invalid memory accesses will result in a heavy penalty.

8 Grading Criteria

Category	Requirement
Honor Code (100%)	**Mandatory for grading! See the Handout Instructions for details.**
Correctness (50%)	- Correct implementation of all required commands. - Correct tree behavior, including parent-child relationships and recursive traversal.
Memory (30%)	- Valgrind/Sanitizer checks pass with 0 leaks. - No invalid reads/writes, use-after-free, or double-free.
Code Quality (10%)	- Modular function design. - Clear variable naming. - Comments explaining complex logic.
Version Control (10%)	- Effective use of Git. - Frequent commits (not just one "final" commit). - Meaningful commit messages describing changes.

Logistics

This is an individual assignment, and all hand-ins are electronic via GitHub. Any clarification or correction will be announced on the KUHub Learn.

Handout Instructions

How to Start

- Accept the GitHub Classroom assignment using the link:
  `https://classroom.github.com/a/5E1Waaro`
- Clone the GitHub repository created for you to a Linux machine in which you plan to do your work (Using Linux servers such as `linuxpool.ku.edu.tr`. See the **How to Use linuxpool.ku.edu.tr Linux Servers** section for details.)

  ```
  $ git clone https://github.com/Comp201-Spring-2026/assignment-2-heap-management-USER
  ```

  (Replace USER with the GitHub username you used to accept the assignment.)
- **[IMPORTANT]** After cloning the repository, create a new file called `honor.txt` containing the following statement, then commit and push it:
  "I hereby declare that I have completed this assignment individually, without support from anyone else."
  You can create the file with:
  `$ echo "I hereby declare that I have completed this assignment individually, without support from anyone else." > honor.txt`
- Then, start filling in the `main.c` file. There is a bare-bones template ready for you to use. If you do not want to use this template, it is fine. The only necessities are clean, not spaghetti code, and the correct outputs.
- You can compile and run your code using the provided Makefile for a specified test case:

  ```
  $ make 1
  ```

Task

Fill in Your Information

Please fill in the comment at the start of the `main.c` file as follows:

```
//  YOUR_NAME,  YOUR_ID
```

Hand-in Instructions

As in previous assignments, submissions are handled through GitHub. We strongly encourage you to use Git effectively by committing frequently and writing meaningful commit messages.

- Commit all the changes you make:

```
$ git commit -a -m "commit message"
(Note: please use meaningful commit messages.)
```

- Push your work to GitHub servers:

```
$ git push origin main
```

Advice

- You will find some hints in the template code. While none of them is required, they are strongly advised since they both will make the assignment easier and will make it easier to get you partial points.
- Keep your GitHub repository by frequently committing the changes.
- Don't forget to use GDB and Valgrind since they can save a lot of time in debugging.
- Use linuxpool.ku.edu.tr Linux servers to test your code in order to avoid compatibility issues.

How to Use linuxpool.ku.edu.tr Linux Servers

- Connect to KU VPN (if you are already connected to the KU network, you can skip this step). See details at: KU VPN Access Guide
- Connect to the `linuxpool.ku.edu.tr` server using SSH (replace USER with your Koç University username):

```
$ ssh USER@linuxpool.ku.edu.tr
(It will ask your password, type your Koc University password.)
```

- When you are finished with your work, you can disconnect by typing:

```
$ exit
```

Your connection to the server may drop sometimes. In that case, you need to reconnect.

We advise you to watch the following video about the usage of SSH, which is used to connect remote servers, and SCP, which is used to transfer files between remote servers and your local machine: SSH and SCP Tutorial

Academic Integrity

All work on assignments must be done individually unless stated otherwise. You are encouraged to discuss the given assignments with your classmates, but these discussions should be carried out in an abstract way. That is, discussions related to a particular solution to a specific problem (either in actual code or in the pseudocode) will not be tolerated. In short, turning in someone else's work, in whole or in part, as your own will be considered as a violation of academic integrity. Please note that the former condition also holds for the web material as everything on the web has been written by someone else. See Koç University - Student Code of Conduct.

Late Submission Policy

You may use up to 7 grace days (in total) over the course of the semester for the assignments. That is, you can submit your solutions without any penalty if you have free grace days left. Any additional unapproved late submission will be punished (1 day late: 20 % off, 2 days late: 40 % off) and **no submission after 2 days will be accepted.**

Regulations

- **KUHub Learn:** This is an individual assignment, and all hand-ins are electronic. Any clarification or correction will be announced on the KUHub Learn.
- **Cheating:** We use automated plagiarism detection to compare your assignment submission with others and also the code repositories on GitHub and similar sites. Moreover, we plan to ask randomly selected 10% of students to explain their code verbally after the assignments are graded. And one may lose full credit if he or she fails from this oral part.